CMPE 321 – Project 2
Part3: Schema Refinement and Normalization

Group 36: Halis Tunay Aslan - Kaan Uz

April 17, 2026

1 Introduction

This report presents the functional dependency analysis and normalization assessment for the TransferDB schema. The database consists of 13 relations designed to manage football players, clubs, matches, transfers, and contracts. Each relation is analyzed for functional dependencies, candidate keys, and compliance with Boyce-Codd Normal Form (BCNF).

2 Functional Dependencies and Normal Form Analysis

2.1 DatabaseManager

Attributes: `username, password_hash`

Functional Dependencies:

- `username` → `password_hash`

Candidate Key: `username`

Normal Form: BCNF. The only non-trivial FD has `username` on the left side, which is the candidate key.

2.2 Person

Attributes: `person_id, username, password_hash, name, surname, nationality, date_of_birth`

Functional Dependencies:

- `person_id` → `username, password_hash, name, surname, nationality, date_of_birth`
- `username` → `person_id, password_hash, name, surname, nationality, date_of_birth`

Candidate Keys: `person_id`, `username`

Normal Form: BCNF. Both FDs have a candidate key on the left side. No partial or transitive dependencies exist.

2.3 Player

Attributes: `person_id, market_value, main_position, strong_foot, height`

Functional Dependencies:

- `person_id` → `market_value, main_position, strong_foot, height`

Candidate Key: `person_id`

Normal Form: BCNF. `person_id` is a superkey and the sole determinant in all non-trivial FDs.

2.4 Manager

Attributes: `person_id, preferred_formation, experience_level`

Functional Dependencies:

- `person_id` → `preferred_formation, experience_level`

Candidate Key: `person_id`

Normal Form: BCNF.

2.5 Referee

Attributes: `person_id, license_level, years_of_experience`

Functional Dependencies:

- `person_id` → `license_level, years_of_experience`

Candidate Key: `person_id`

Normal Form: BCNF.

2.6 Stadium

Attributes: `stadium_id, stadium_name, city, capacity`

Functional Dependencies:

- `stadium_id` → `stadium_name, city, capacity`
- {`stadium_name, city`} → `stadium_id, capacity`

Candidate Keys: `stadium_id`, `(stadium_name, city)`

Normal Form: BCNF. Both FDs have a candidate key on the left side. The composite candidate key `(stadium_name, city)` reflects that the same stadium name may exist in different cities.

2.7 Club

Attributes: `club_id, club_name, city, foundation_year, manager_id, stadium_id`

Functional Dependencies:

- club_id → club_name, city, foundation_year, manager_id, stadium_id
- club_name → club_id, city, foundation_year, manager_id, stadium_id
- manager_id → club_id (when non-null; each manager manages at most one club)

Candidate Keys: club_id, club_name

Normal Form: BCNF. The only non-trivial FDs have club_id and club_name as determinants, both of which are candidate keys. Since manager_id is constrained UNIQUE, any non-null manager_id uniquely identifies a club, acting as an alternate candidate key. Therefore the dependency manager_id → club_id also has a candidate key on the left-hand side, and BCNF is not violated.

2.8 Competition

Attributes: competition_id, name, season, country, competition_type

Functional Dependencies:

- competition_id → name, season, country, competition_type
- {name, season} → competition_id, country, competition_type

Candidate Keys: competition_id, (name, season)

Normal Form: BCNF. Both determinants are candidate keys.

2.9 Match

Attributes: match_id, competition_id, home_club_id, away_club_id, stadium_id, referee_id, match_datetime, attendance, home_goals, away_goals

Functional Dependencies:

- match_id → competition_id, home_club_id, away_club_id, stadium_id, referee_id, ma

Candidate Key: match_id

Normal Form: BCNF. All attributes are directly determined by the primary key. attendance, home_goals, and away_goals are nullable (unknown until the referee submits results), which is a valid design choice rather than a normalization issue.

2.10 Contract

Attributes: contract_id, player_id, club_id, start_date, end_date, weekly_wage, contract_type

Functional Dependencies:

- contract_id → player_id, club_id, start_date, end_date, weekly_wage, contract_typ

Candidate Key: contract_id

Normal Form: BCNF. A single candidate key determines all other attributes with no partial or transitive dependencies.

2.11 Transfer_Record

Attributes: transfer_id, player_id, from_club_id, to_club_id, transfer_date, transfer_fee, transfer_type

Functional Dependencies:

- transfer_id → player_id, from_club_id, to_club_id, transfer_date, transfer_fee, t

Candidate Key: transfer_id

Normal Form: BCNF. from_club_id is nullable (a player may join a club as a free agent with no previous club), which does not affect normalization.

2.12 Match_Squad

Attributes: player_id, match_id, club_id, is_starter

Functional Dependencies:

- {player_id, match_id} → club_id, is_starter

Candidate Key: (player_id, match_id)

Normal Form: BCNF. The composite primary key is the only determinant. There are no partial dependencies since the relation has only a composite key.

2.13 Match_Stats

Attributes: player_id, match_id, club_id, is_starter, minutes_played, position_in_matc goals, assists, yellow_cards, red_cards, rating

Functional Dependencies:

- {player_id, match_id} → club_id, is_starter, minutes_played, position_in_match, g

Candidate Key: (player_id, match_id)

Normal Form: BCNF. All non-key attributes are fully functionally dependent on the composite primary key. No transitive dependencies exist among the statistical attributes. Although club_id and is_starter could be derived from Match_Squad, they are intentionally stored here for query performance and snapshot consistency — this is a controlled denormalization and does not violate BCNF.

3 Summary

Relation	Candidate Key(s)	Normal Form
DatabaseManager	username	BCNF
Person	person_id, username	BCNF
Player	person_id	BCNF
Manager	person_id	BCNF
Referee	person_id	BCNF
Stadium	stadium_id, (stadium_name, city)	BCNF
Club	club_id, club_name	BCNF
Competition	competition_id, (name, season)	BCNF
Match	match_id	BCNF
Contract	contract_id	BCNF
Transfer_Record	transfer_id	BCNF
Match_Squad	(player_id, match_id)	BCNF
Match_Stats	(player_id, match_id)	BCNF

Table 1: Normal form summary for all relations

4 Additional Constraint Enforcement

Certain domain-specific constraints could not be captured through functional dependencies alone and were enforced at the database level using SQL `CHECK` constraints, triggers, and stored procedures.

CHECK constraints enforce domain integrity on individual attributes: position values, strong foot options, contract types, transfer types, competition types, rating range (1.0–10.0), yellow cards (0–2), minutes played (0–120), and non-negative wages and fees.

Triggers enforce multi-row and cross-table business rules that cannot be expressed as simple constraints:

- No stadium, club, or referee may be involved in two matches within 120 minutes of each other (`trg_match_schedule_check`, `trg_match_datetime_update`).
- Match results and player statistics can only be submitted after the scheduled match time has passed (`trg_match_result_check`, `trg_stats_insert`).
- A player may hold at most one active Permanent and one active Loan contract simultaneously; a Loan requires an active Permanent at a different club (`trg_contract_insert`, `trg_contract_update`).
- Each club may register at most 23 players and 11 starters per match; loan players cannot appear for their parent club (`trg_squad_insert`, `trg_squad_update`, `trg_stats_update`).
- A manager may manage at most one club at a time (`trg_manager_unique_club`).
- Stadium capacity cannot be reduced below the attendance of any past match held there (`trg_stadium_capacity_update`).

Stored procedures encapsulate multi-step operations that require transactional consistency:

- `sp_register_transfer` registers a transfer and new contract, automatically terminating any existing Permanent contract.
- `sp_submit_match_result` enforces referee authorization at the database level before updating match scores and attendance.
- `sp_finalize_squad` validates the minimum squad size of 11 players at the database level after squad submission.

5 Conclusion

All 13 relations in the TransferDB schema are in Boyce-Codd Normal Form. In every relation, the left-hand side of each non-trivial functional dependency is a superkey, satisfying the BCNF condition. Since no relation violates BCNF, the evaluation of 3NF was not required. No decomposition was necessary. The schema was designed with normalization in mind from the start, using surrogate primary keys where natural keys were composite or unstable, and separating entity subtypes (Player, Manager, Referee) from the supertype (Person) to avoid redundancy. Since all relations were inherently in BCNF and no decomposition was required, the criteria for lossless-join and dependency preservation are naturally satisfied by the original schema design.